Archivara Research Pipeline via Codex CLI

Archivara Agent: Local Cellular-Automaton Escape from the Radius-Limited Locality Barrier on the Hadamard-668 Frontier

March 18, 2026

Repository scope. The initiating request asked whether an untried "cellar automata" method could solve Hadamard order 668. No distinct method family of that name appears in the repository or the literature snapshot, so throughout the paper we interpret the phrase as *cellular automata*. The exact target remains an order-668 real Hadamard matrix. No such matrix is found here. What the repository does contain, however, is a complete cellular-automaton program with one clear negative result, one exact finite locality barrier, one exact local counterexample class, and two transfer branches that reproduce the same non-exact frontier improvement.

Artifact summary. The manuscript is written directly from the current contents of `results/`, `figures/`, the root bibliography `sources.bib`, and the post-H1 commit range `18a1568` through `5cf94fa`. The direct quantitative anchors are the seed manifest `results/frontier/order_668_64m/seed_manifest.json`, the locality atlas `results/analysis/composite_packet_locality_atlas.json`, the barrier certificate `results/verification/radius_limited_locality_barrier.json`, the hypergraph/orbit/lattice-gas/population summaries under `results/experiments/`, and the family-leakage audit `results/verification/family_leakage_audit.json`.

Abstract

We study the exact real Hadamard problem at order $668 = 4 \cdot 167$ from the strongest in-repository frontier anchor currently available: Eliahou's 2025 64-modular order-668 near-solution, recovered as a compact pair of length-167 binary sequences. The prompt asked for a cellular-automaton approach. The repository therefore tests, and then progressively redesigns, local cellular-automaton dynamics on that published frontier seed under a shared exactness harness.

The first result is negative. The executed one-packet defect-syndrome cellular automaton `H1_defect_syndrome_ca_64m`, benchmarked against greedy search, tabu search, simulated annealing, and stochastic hillclimbing, reaches exactness on two tiny solved controls but neither reaches exact orthogonality nor improves the canonical frontier objective $(13, 2880, 512)$. An exhaustive exact delta scan then sharpens the failure: all 334 one-packet moves, all 55,611 unordered two-packet moves, and all 9 retained low-splash composite actions fail as depth-1 single actuators. The retained library reduces the median changed-lag footprint from 51 to 9, but still yields no improving single action. This is an exact locality barrier, not merely a failed pilot.

The second result is positive but narrow. The same barrier certificate finds the first verified escape at radius 1, depth 2 on the retained overlap graph. The hyperedge $[3, 7]$, corresponding to the action sequence $[\texttt{q[53]}, \texttt{q[136]}]$ followed by $[\texttt{q[29]}, \texttt{s[29]}, \texttt{q[114]}, \texttt{s[114]}]$, produces the six-packet state $(13, 2744, 480)$, improving the frontier seed by 136 in ℓ_1 defect and by 32 in maximum defect while preserving support 13. A lag-core lattice-gas automaton rederives the same state through an explicit continuity law and passes a Williamson/Turyn/Goethals-Seidel/cocyclic/block-circulant leakage audit. A symmetry-quotient orbit automaton with one fixed 137-signature rule table transfers the same improvement across the canonical seed and five frontier perturbation states. A stochastic population automaton does not separate from its zero-coupling ablation and is retired honestly.

No exact order-668 Hadamard matrix is found. The contribution is instead a precise structural answer to the cellular-automaton question. One-packet CA locality at order 668 is fake-local and blocked at depth 1, but actuator redesign plus short-horizon local coordination produces a reproducible, leakage-audited frontier improvement. Cellular automata do not solve Hadamard 668 here, yet they do expose a concrete local mechanism that survives beyond the original H1 falsification.

1 Introduction

An $n \times n$ real Hadamard matrix is a ± 1 matrix H satisfying

$$HH^\top = nI_n. \tag{1}$$

The Hadamard existence problem is both elementary to state and difficult to close. Large construction catalogues, modular variants, and certificate-rich structured searches coexist with stubborn open orders [2, 3, 5]. The present repository studies one such order: $668 = 4 \cdot 167$.

The exact target is not a modular surrogate. It is an exact real Hadamard matrix of order 668. The strongest seed available in-repository is instead Eliahou's 2025 64-modular Hadamard matrix of order 668, encoded compactly by two length-167 sign sequences with exactly 13 nonzero combined correlation coefficients [6]. The project therefore begins from a published near-solution, not from an exact witness.

The user asked for a method that had not yet been tried and named it "cellar automata." The repository contains no independent technical meaning for that phrase, so the only coherent interpretation is *cellular automata*. That interpretation immediately forces two standards of honesty.

1. A CA result does not count as a solution unless exact orthogonality is reached.
2. A CA result does not count as a new method contribution merely because local-rule language is used; the update geometry must do something materially different from direct scoring or familiar design-generation patterns.

The history of this repository now contains both a failure and a follow-up. The original H1 branch, a one-packet defect-syndrome CA in the recovered q/s coordinates, fails cleanly under matched controls. The later retained-library branches, added after the H1 draft paper but before the current writing pass, do *not* solve order 668 either. They do, however, sharpen the answer in a way the stale draft does not: they certify a depth-1 locality barrier and then exhibit the first local counterexample class that crosses it.

The resulting paper is therefore not a frontier-solution paper. It is a barrier-and-counterexample paper on a concrete open order. Its contributions are the following.

C1. We formalize the recovered order-668 frontier seed, the shared exactness harness, and the exact packet/composite delta algebra used throughout the repository.

C2. We preserve the original H1 result as a rigorously benchmarked negative control: H1 solves tiny exact controls but neither reaches exactness nor improves the canonical frontier seed.

C3. We exhaustively enumerate a retained low-splash composite actuator library and prove, by exact finite computation, that all depth-1 single actuators in the checked class are blocked on the canonical seed.

C4. We certify the first escaping local rule class: a radius-1, depth-2 retained hyperedge rule whose first counterexample reaches the six-packet state $(13, 2744, 480)$.

C5. We show that the same non-exact frontier improvement is reproduced by a lag-core lattice-gas CA with an explicit continuity law and by a symmetry-quotient orbit CA with a single fixed rule table.

C6. We show that a population self-stabilizing CA does not establish a robustness advantage over zero-coupling and therefore does *not* justify a stronger self-organization claim.

The paper is organized as follows. Section 2 positions the work against modular Hadamard constructions, exact and heuristic search, and CA-adjacent design literature. Section 3 defines the order-668 representation, the objective, the retained action graph, and the orbit signatures. Section 4 gives the complete algorithmic pipeline, including pseudocode and complexity analysis. Section 5 states and proves the exact delta identities, the continuity law, the locality barrier, and the counterexample theorem. Section 6 documents all experiment inputs, budgets, seeds, and reproducibility limits. Section 7 reports the negative H1 evidence, the retained-library barrier, the post-H1 branch outcomes, and the failed population robustness test. Section 8 explains what the results establish, what remains open, and where the current evidence still stops short.

2 Related Work

2.1 Order 668 and the modular frontier

The relevant frontier anchor is precise. Eliahou and Kervaire developed the modular-sequence framework and its relation to modular Hadamard matrices [4, 5]. Eliahou's 2025 order-668 construction then supplies the exact near-solution used in every frontier experiment of this paper: a compact length-167 seed whose derived Goethals-Seidel lift is Hadamard modulo 64 and whose nonzero-lag defect profile contains exactly 13 exceptional coefficients [6]. Our paper does not claim a new order-668 seed, a new modular theorem, or a new exact witness. The Eliahou paper is the provenance source and the novelty ceiling.

2.2 Exact and heuristic Hadamard search

Exact combinatorial search remains the strongest evidence standard. Bright and collaborators show how SAT+CAS methods can certify exhaustive search statements for structured Hadamard-adjacent families such as Williamson and best matrices [1, 2]. This line matters here mainly as a contrast: our repository returns no exact order-668 witness and no proof of nonexistence. The present work is heuristic and local, not certificate-rich exact search.

Within heuristic search, Suksmono and collaborators study simulated quantum annealing, quantum annealing, and related optimization formulations for Hadamard search [9–11]. These are the relevant comparator papers for any broad statement about search competitiveness. We therefore avoid such statements. The saved repository evidence is strong enough to eliminate branches, but not to rank CA against the full annealing literature.

2.3 Cellular automata as search and design machinery

Cellular automata are not new in the most general sense. Tsompanas et al. use CA as a distributed local-search mechanism for the shortest path problem [12]. That paper is not Hadamard-specific, but it blocks any claim that "CA for search" is itself novel. CA also appear in adjacent combinatorial-design settings: mutually orthogonal Latin squares from CA [8] and bent-function constructions from CA [7]. Those papers matter for a different reason. They show that CA can contribute structurally to combinatorial design, but they also show that our claim must stay narrow. We are not presenting a new CA construction of a Hadamard family. We are studying one seed-specific repair program and the locality structure it induces.

2.4 How the present work differs

The repository's prior-art gap file is explicit about the limits of novelty. Relative to Eliahou's 2025 order-668 paper, we contribute only downstream search dynamics on top of the published seed. Relative to Tsompanas et al., we are not making a general CA-for-optimization claim, but a specific locality claim on one open Hadamard instance. Relative to Suksmono's Hadamard-search heuristics and Bright's exact-search line, we are narrower in scope and weaker in certification. Relative to the CA-based design-construction papers, we are not generating a new combinatorial family. The surviving contribution is therefore exact and local: an explicit barrier/counterexample

pair for cellular-automaton search on the published order-668 frontier seed.

3 Background and Preliminaries

3.1 Recovered order-668 seed

Let $\ell = 167$. The repository stores the frontier seed as two sequences

$$q, s \in \{\pm 1\}^{\ell}, \tag{2}$$

decoded from Eliahou's published run-length encoding. Indices always run from 0 to $\ell - 1$.

Definition 3.1 (Prime involution)**.** *For $x \in \{\pm 1\}^{\ell}$, define*

$$x'_i = \begin{cases} x_i, & 0 \le i < (\ell+1)/2, \\ -x_i, & (\ell+1)/2 \le i \le \ell - 1. \end{cases} \tag{3}$$

Define the pointwise product $(q \odot s)_i = q_i s_i$. The shared harness uses the derived quadruple

$$A = s, \qquad B = s', \qquad C = q \odot s, \qquad D = (q \odot s)'. \tag{4}$$

The seed manifest `results/frontier/order_668_64m/seed_manifest.json` and source excerpt `results/frontier/order_668_64m/source_excerpt.txt` verify that this lift reproduces exactly the published 13 exceptional coefficients.

Definition 3.2 (Aperiodic autocorrelation)**.** *For $x \in \{\pm 1\}^{\ell}$ and lag $k \in \{0, 1, \ldots, \ell - 1\}$, define*

$$N_x(k) = \sum_{t=0}^{\ell-k-1} x_t x_{t+k}. \tag{5}$$

The combined coefficient vector is

$$c_k(q, s) = N_A(k) + N_B(k) + N_C(k) + N_D(k), \qquad k = 0, \ldots, \ell - 1. \tag{6}$$

By construction $c_0 = 4\ell = 668$.

Definition 3.3 (Primary defect tuple)**.** *For a state (q, s), define*

$$\operatorname{support}(q, s) = \left|\{k \in \{1, \ldots, \ell - 1\} : c_k(q, s) \neq 0\}\right|, \tag{7}$$

$$\ell_1(q, s) = \sum_{k=1}^{\ell-1} |c_k(q, s)|, \tag{8}$$

$$\max(q, s) = \max_{1 \le k \le \ell-1} |c_k(q, s)|. \tag{9}$$

The scalar proxy used internally for search is

$$\operatorname{score}(q, s) = 10^6 \operatorname{support}(q, s) + 10^3\, \ell_1(q, s) + \max(q, s). \tag{10}$$

The canonical frontier seed starts at

$$\big(\operatorname{support}, \ell_1, \max\big) = (13, 2880, 512), \tag{11}$$

with nonzero lags exactly at

$$\{4, 8, 12, 16, 26, 30, 34, 38, 42, 46, 50, 54, 58\}. \tag{12}$$

Figure 1 visualizes the recovered sequences and the combined correlation profile.

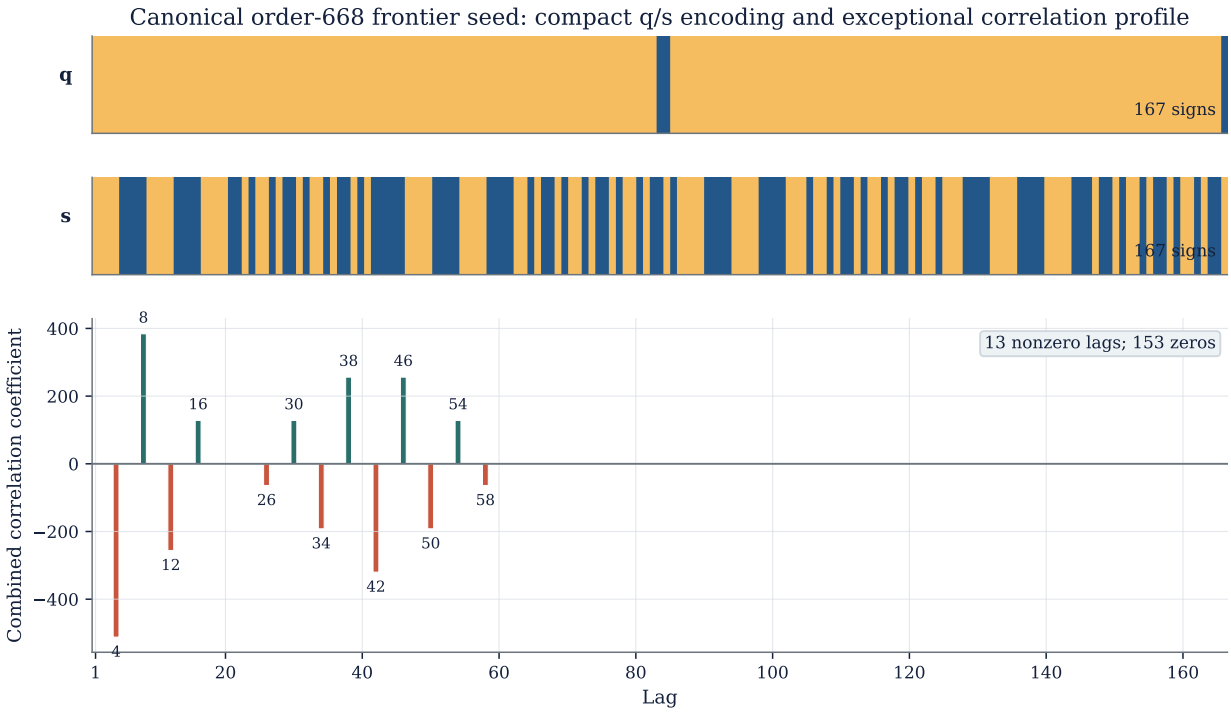

Figure 1: **Recovered order-668 frontier seed.** The upper strips show the exact binary q and s sequences decoded from Eliahou's published run-length encoding. The lower panel shows the combined coefficients from (6). Only 13 of the 166 nonzero-lag coefficients are nonzero, and their support is exactly the fixed set in (12). This figure is the starting object for every order-668 experiment in the paper; it is not itself a search result produced here.

3.2 Packets, composite actions, and retained actions

The raw packet basis is

$$\mathcal{P} = \{0, 1, \ldots, 2\ell - 1\}. \tag{13}$$

Packets 0 through $\ell - 1$ flip one coordinate of q, while packets ℓ through $2\ell - 1$ flip one coordinate of s. For a packet set $I \subseteq \mathcal{P}$, application is parity-based: flipping the same packet twice cancels.

Definition 3.4 (Composite action)**.** *A composite action is a finite packet multiset, interpreted modulo parity. Its active q- and s-toggle sets are the odd-multiplicity packet indices in each channel.*

The composite-library scan enumerates five explicit actuator families around the canonical seed:

(i) all unordered packet pairs,

(ii) all channel-complete four-packet actions $\{q[i], s[i], q[j], s[j]\}$ with $i < j$,

(iii) all matched boundary quads on run boundaries,

(iv) all s-boundary pairs,

(v) all q-boundary pairs.

The direct one-packet median changed-lag footprint is 51, so the post-H1 pipeline defines the *low-splash cutoff* as 25.5, exactly half of that median. Inside the low-splash composite set, the repository retains the Pareto frontier over

$$(\text{changed\_lag\_count},\ \text{support},\ \ell_1,\ \max).$$

This produces a retained library

$$\mathcal{R} = \{a_0, \ldots, a_8\} \tag{14}$$

of 9 actions with median changed-lag count 9.

3.3 Retained-state graph and causal cones

Each retained action $a_i \in \mathcal{R}$ has a changed-lag set $\Gamma(a_i) \subseteq \{1, \ldots, \ell - 1\}$. The retained-state graph is built by XOR-combining subsets of the retained actions. Since $|\mathcal{R}| = 9$, there are at most $2^9 = 512$ subset masks. The constructor in `hadamard_ca/retained_state_graph.py` exhausts all 512 masks and, in the actual artifact, finds 512 unique parity states.

Definition 3.5 (Retained state graph)**.** *The retained state graph $G_{\mathcal{R}} = (\mathcal{S}, \mathcal{R}, E)$ has*

- *one state $s \in \mathcal{S}$ for each unique parity packet mask induced by a subset of $\mathcal{R}$,*
- *one immediate transition $s \to s \oplus a_i$ for each retained action a_i,*
- *one pairwise neighborhood edge between a_i and a_j if $\Gamma(a_i) \cap \Gamma(a_j) \neq \varnothing$,*
- *one* causal-cone *hyperedge (a_i, a_j) when the pairwise-overlap condition holds and*

$$|\Gamma(a_i) \cup \Gamma(a_j)| \leq 25.$$

The graph constructor stores the exact coefficient vector and objective tuple for every retained state, so the runtime branches never need to rescan the original 334 packet basis once the retained graph is built.

3.4 Lag field and orbit signatures

For any retained state, define the signed lag-defect field

$$\rho(k) = c_k, \qquad k = 1, \ldots, \ell - 1. \tag{15}$$

The lattice-gas branch works directly on the transition signature of ρ, while the orbit branch quotients candidate events by the tuple

$$\sigma(e) = \big(\kappa(e), a(e), u(e), b(e), t(e), r(e)\big), \tag{16}$$

where

- $\kappa(e)$ is the normalized event kind (`annihilation`, `move`, or `split`),
- $a(e) \in \{1, 2\}$ is the action count,

Table 1: **Core notation.**

Symbol	Meaning
n	target Hadamard order, here 668
ℓ	core sequence length, here 167
q, s	recovered frontier seed sequences in $\{\pm 1\}^{\ell}$
A, B, C, D	derived quadruple from (4)
c_k	combined coefficient at lag k from (6)
$\mathcal{P}$	raw one-packet actuator basis of size 334
$\mathcal{R}$	retained low-splash action library of size 9
$\mathcal{S}$	retained parity-state set of size 512
$\Gamma(a)$	changed-lag support of action a
ρ	signed lag-defect field on $\{1, \ldots, 166\}$
J	nearest-neighbor transport current in the lattice-gas formulation
$\sigma(e)$	symmetry-quotient orbit signature from (16)

- $u(e) \in \{\texttt{decrease}, \texttt{conserve}, \texttt{increase}\}$ is the support-flux class,
- $b(e) \in \{\texttt{1}, \texttt{2}, \texttt{3\_4}, \texttt{5\_8}, \texttt{9\_plus}\}$ is the changed-lag-count bin,
- $t(e) \in \{\texttt{tight}, \texttt{mid}, \texttt{wide}\}$ is the transport-span bin,
- $r(e)$ is the canonical sign-run count of the delta profile.

By construction $\sigma(e)$ discards raw cyclic coordinates, is invariant under reversal, and ignores global q/s sign orientation.

4 Method

4.1 Shared harness and exactness contract

All methods report through the same harness in `hadamard_ca/harness.py`. For every run, the harness computes

- exact-hit status (`exact_hit`),
- correlation-defect support,
- off-diagonal Gram-defect support and maximum magnitude,
- the objective tuple from Definition 3.3,
- wall time, objective-evaluation counts, restart counts, and a sign-quotiented fingerprint.

The harness is validated on exact controls of lengths 5 and 7 and is reused unchanged for the order-668 seed and the later retained-basis branches. The exactness gate is therefore identical throughout the paper.

4.2 Negative-control stack: H1 and same-coordinate baselines

The original H1 branch remains important because it is the control that failed cleanly enough to motivate the later redesign.

H1 defect-syndrome CA. The H1 state is

$$(q, s, c, \mathcal{M}, r),$$

where $q, s \in \{\pm 1\}^{167}$, c is the exact coefficient vector, $\mathcal{M}$ is a radius-1 active-lag neighborhood around the current defect support, and $r \in \mathbb{Z}_{\geq 0}^{334}$ is packet-level refractory memory. One H1 accepted-state update does the following.

1. recompute the exact coefficient vector c,
2. build the active-lag mask $\mathcal{M}$,
3. evaluate the exact one-packet delta Δ_p for each $p \in \mathcal{P}$,
4. convert Δ_p into a raw packet pressure with active-lag bonuses and spill penalties,
5. smooth raw pressure over a packet graph with radius-2 same-channel edges and same-site cross-channel edges,
6. subtract refractory penalties and select locally maximal packets above threshold,
7. apply at most one packet, update the refractory field, and stop on exactness, stagnation, or budget exhaustion.

The frontier H1 configuration is the saved one from `results/experiments/order_668_64m/configs/H1_defect_syndrome_ca_64m.json`: lag radius 1, packet radius 2, same-channel weight 0.15, cross-channel weight 0.35, activation threshold 0.5, refractory steps 2, stagnation limit 8, requested restarts 3, and restart packet flips 2.

Same-coordinate non-CA baselines. Greedy search, tabu search, simulated annealing, and stochastic hillclimbing all operate on the same one-packet basis, the same q/s state space, and the same scalar score (10). This matters because the H1 negative result is then attributable to the update rule rather than to a privileged state representation.

Complexity of H1. The exact coefficient recomputation and all-packets delta sweep dominate. With fixed neighborhood radii, one accepted-state H1 update costs $O(\ell^2)$ arithmetic and $O(\ell)$ working memory beyond the trace, exactly as in the original H1 paper draft.

4.3 Algorithm 1: exhaustive actuator atlas and retention rule

The post-H1 pass does not begin by tuning the H1 weights. It first changes the actuator basis.

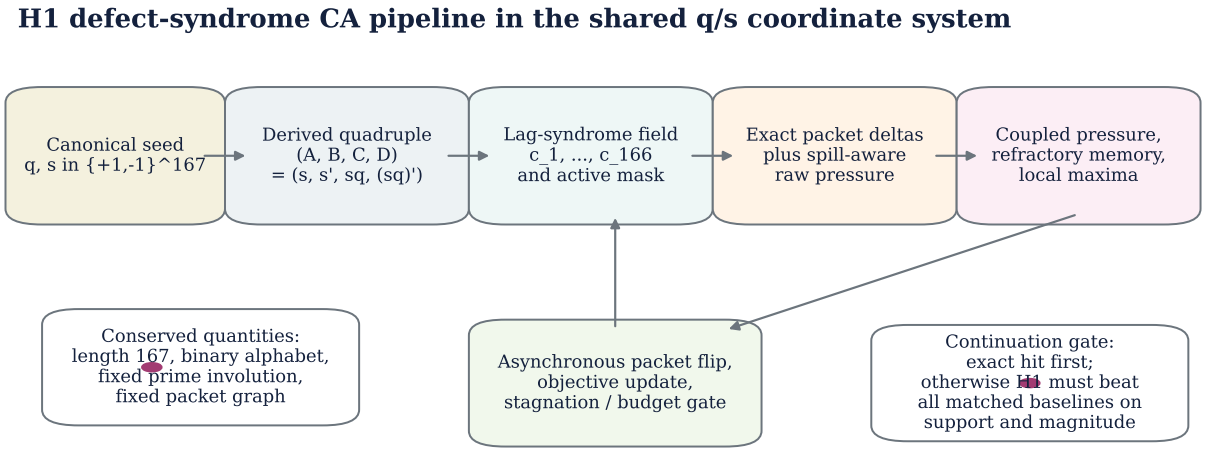

Figure 2: **Original H1 pipeline, preserved as the negative control.** H1 works directly on the recovered q/s seed coordinates, not on the full 668×668 sign lattice. Exact one-packet deltas are scored over the whole packet basis, then smoothed by same-channel and cross-channel coupling plus refractory memory. The later retained-basis branches do not retune this rule; they replace its actuator library and locality geometry.

Enumeration. Using the exact composite delta identity proved in Section 5, the script `scripts/generate_composite_locality_atlas.py` evaluates

- 55,611 unordered packet pairs,
- 13,861 channel-complete balanced-four actions,
- 85 matched boundary quads,
- 84 s-boundary pairs,
- 3 q-boundary pairs,

for a total of 69,644 composite candidates on the canonical frontier seed, plus a harder control cross-check.

Retention. The low-splash cutoff is fixed at 25.5, exactly half the one-packet median changed-lag footprint 51. Inside the frontier low-splash set, the retained library is the Pareto frontier in changed-lag count and the three components of the defect tuple. This produces nine actions. Seven are pairs, two are balanced-four actions, and none individually improves the frontier seed.

Complexity. Let M be the number of candidate composite actions and let r be the maximum number of toggled coordinates in any candidate. After precomputing the single-site delta table, one candidate evaluation uses $O(\ell + r^2)$ time. Here $r \leq 4$, so the entire atlas construction is $O(M\ell)$ with a small constant.

4.4 Algorithm 2: retained-state hypergraph CA

Once the retained library is fixed, the hypergraph CA no longer scans the original 334 packets at runtime.

State graph. The retained-state graph has 9 actions, 512 states, 9 immediate transitions from each state, and 56 ordered two-step causal cones satisfying the retained overlap rule in Definition 3.5. The canonical run uses a lookup budget of 256 graph transitions.

Immediate baselines. The same retained basis supports four depth-1 controls:

(a) `pairwise_graph_ca`: coupled immediate search with one-step refractory,

(b) `zero_coupling`: immediate lexicographic search without coupling,

(c) `zero_refractory`: coupled immediate search without refractory blocking,

(d) `scorer_only`: uncoupled, unrefracted immediate search.

Hypergraph rule. At each state, the hypergraph CA scans all admissible causal cones rooted at the local neighborhood, computes the exact end-state objective for each cone using the precomputed state graph, and accepts the lexicographically best improving cone if one exists. The canonical run accepts exactly one cone, then stops when the next scan shows no improving depth-2 move.

Runtime. With the retained graph precomputed, one full immediate scan is $O(|\mathcal{R}|)$ graph lookups and one full hypergraph scan is $O(|\mathcal{C}|)$ graph lookups, independent of the original 668×668 matrix. In the actual artifact, the canonical hypergraph run spends 210 transition lookups and never rescans the raw packet basis.

4.5 Algorithm 3: defect-charge lattice-gas CA

The lattice-gas branch reinterprets the same retained basis in lag space. It works with the signed lag field ρ and the transport signature induced by a candidate transition.

Candidate generation. The branch first evaluates all single retained actions. If any direct single action improves the current objective, it chooses among those. Otherwise it opens the precomputed two-step carrier cones. The candidate factory is therefore

singles first, then carriers only when singles are blocked.

Ranking. For each candidate, the branch computes

$$(\text{support flux},\ \text{reservoir magnitude},\ \text{transport span},\ \text{warning overlap},\ -\ell_1 \text{ gain},\ -\text{max gain}).$$

Single actions are always admissible if they improve. Two-step carriers are admissible only when

$$\text{support flux} \le 0, \qquad |R| \le 16, \qquad \ell_1 \text{ gain} > 0. \tag{17}$$

This rule is intentionally conservative: temporary support diffusion is allowed only inside a carrier macro-event whose final state returns to nonincreasing support.

Controls. Two matched lag-space controls are run on the same retained graph:

(a) `lag_greedy_control`, which chooses the best improving single retained action,

(b) `warning_field_control`, which preserves warning-memory bookkeeping but still restricts itself to single actions.

4.6 Algorithm 4: orbit-quotient CA

The orbit CA removes raw coordinate dependence. For any state, it groups all improving single actions and two-step carriers by the signature (16). Each signature class contributes exactly one representative event to the runtime decision set.

Training mix. The saved orbit rule table has 137 signatures. It is trained from

- 24 states from `control_n5_q0` with 162 improving representatives,
- 24 states from `control_n7_q0` with 338 improving representatives,
- 64 states from `control_n9_hardest_pair` with 2259 improving representatives,
- four predefined frontier perturbation states with counts $3, 3, 1, 1$.

The canonical frontier seed itself is *not* part of training.

Runtime rule. At each state, the orbit CA forms the representative set, filters to improving representatives, and chooses the minimum event under

$$(\text{rule priority},\ \text{fallback orbit rank},\ \text{lattice-gas rank}).$$

The canonical frontier run reaches the improved state in 65 representative lookups.

4.7 Algorithm 5: population self-stabilizing CA

The final branch keeps the orbit representation but adds a stochastic population layer.

Rule table. Unlike the orbit CA, the population branch uses a *control-only* rule table of size 133. It is trained on non-frontier controls only.

Dynamics. At each state, the algorithm:

1. computes one base score per orbit representative,
2. adds coupling-weighted neighbor mass,
3. samples a population of votes through a Gumbel-softmax distribution,
4. updates a memory distribution over orbit representatives,
5. accepts the top representative only if it both improves the current score and exceeds a contraction threshold.

The selected operating point is population size 64, coupling strength 1.6, temperature 1.2, contraction threshold 0.42, memory mix 0.55, maximum rounds per step 6, and RNG seeds $\{11, 13, 17, 19, 23\}$. The zero-coupling ablation sets the coupling strength to 0 and leaves every other parameter unchanged.

5 Theoretical Results and Proofs

The theorems in this section do not prove Hadamard existence at order 668. They instead certify the exact delta machinery, the lattice-gas continuity law, the depth-1 locality barrier, and the first escaping local rule class.

Proposition 5.1 (Exact single-entry autocorrelation delta)**.** *Let $x \in \{\pm 1\}^\ell$ and let $x^{(i)}$ denote the sequence obtained by flipping only coordinate i. Then for every lag $k \in \{1, \dots, \ell - 1\}$,*

$$N_{x^{(i)}}(k) - N_x(k) = -2x_i x_{i+k}\mathbf{1}\{i + k < \ell\} - 2x_{i-k}x_i\mathbf{1}\{i - k \geq 0\}. \tag{18}$$

Proof. Fix a lag $k \geq 1$. By Definition 3.2,

$$N_x(k) = \sum_{t=0}^{\ell-k-1} x_t x_{t+k}.$$

We compare each summand in $N_x(k)$ with the corresponding summand in $N_{x^{(i)}}(k)$.

If neither t nor $t + k$ equals i, then the tth summand is unchanged, because x and $x^{(i)}$ differ only at coordinate i.

There are at most two changed summands.

Case 1: $t = i$. This case occurs exactly when $i + k < \ell$. The original summand is $x_i x_{i+k}$, while the new summand is $(-x_i)x_{i+k}$. Their difference is

$$(-x_i)x_{i+k} - x_i x_{i+k} = -2x_i x_{i+k}.$$

Case 2: $t = i - k$. This case occurs exactly when $i - k \geq 0$. The original summand is $x_{i-k}x_i$, while the new summand is $x_{i-k}(-x_i)$. Their difference is

$$x_{i-k}(-x_i) - x_{i-k}x_i = -2x_{i-k}x_i.$$

No other summand changes. Summing the contributions from the two cases gives (18). This proves the proposition. □

Theorem 5.2 (Exact multi-toggle delta for a single sequence)**.** *Let $x \in \{\pm 1\}^\ell$ and let $I \subseteq \{0, \dots, \ell - 1\}$ be a finite set of toggled coordinates. Let $x^{(I)}$ be obtained by flipping every coordinate in I. For each lag $k \in \{1, \dots, \ell - 1\}$,*

$$N_{x^{(I)}}(k) - N_x(k) = \sum_{i \in I} \delta_x(i, k) + 4 \sum_{\substack{i<j \\ i,j \in I \\ j-i=k}} x_i x_j, \tag{19}$$

where $\delta_x(i, k)$ is the right-hand side of (18).

Proof. Fix a lag $k \geq 1$. For every valid pair $(t, t + k)$, define the original contribution

$$u_t = x_t x_{t+k}.$$

After flipping all coordinates in I, the same pair contributes

$$u_t^{(I)} = \eta_t \eta_{t+k} x_t x_{t+k},$$

where

$$\eta_r = \begin{cases} -1, & r \in I, \\ 1, & r \notin I. \end{cases}$$

Therefore the pair contribution changes by

$$u_t^{(I)} - u_t = (\eta_t \eta_{t+k} - 1) x_t x_{t+k}.$$

There are three possibilities.

(i) If neither endpoint belongs to I, then $\eta_t \eta_{t+k} = 1$ and the pair contributes 0.

(ii) If exactly one endpoint belongs to I, then $\eta_t \eta_{t+k} = -1$ and the pair contributes $-2x_t x_{t+k}$.

(iii) If both endpoints belong to I, then $\eta_t \eta_{t+k} = 1$ and the pair again contributes 0.

Now consider the sum of the single-entry deltas $\sum_{i \in I} \delta_x(i, k)$. By Proposition 5.1, each toggled endpoint in a valid lag-k pair contributes $-2x_t x_{t+k}$ to this sum. Hence:

(a) a pair with exactly one toggled endpoint contributes exactly $-2x_t x_{t+k}$ to the single-entry sum, which matches the true multi-toggle change;

(b) a pair with both endpoints toggled contributes $-4x_t x_{t+k}$ to the single-entry sum, even though the true multi-toggle change is 0.

Thus the single-entry sum is correct for all pairs except the pairs whose two endpoints are both toggled. For such a pair, say (i, j) with $i < j$ and $j - i = k$, the overcount is exactly $-4x_i x_j$, so the correction needed is $+4x_i x_j$. Summing this correction over all toggled pairs at distance k yields the second term in (19). No further correction is necessary, because every lag-k pair contains at most two endpoints. Therefore (19) holds for each k, proving the theorem. □

Corollary 5.3 (Exact composite-action coefficient delta)**.** *Let a composite action toggle the q-indices I_q and the s-indices I_s, and let*

$$I_{qs} = I_q \triangle I_s$$

be their symmetric difference. Then the exact coefficient delta for the combined vector (6) is

$$\Delta = \Delta_s(I_s) + \Delta_{s'}(I_s) + \Delta_{q\odot s}(I_{qs}) + \Delta_{(q\odot s)'}(I_{qs}), \tag{20}$$

where each $\Delta_x(I)$ is the lag vector from Theorem 5.2 applied to sequence x and toggle set I.

Proof. Flipping a subset of q coordinates does not change s or s', while flipping a subset of s coordinates changes both s and s' on exactly those coordinates. For the product sequence $q \odot s$, a coordinate changes sign exactly when an odd number of the corresponding q and s coordinates are toggled, which is precisely the symmetric-difference condition $I_q \triangle I_s$. The same statement holds for $(q \odot s)'$ because the prime involution multiplies coordinates by a fixed sign mask and therefore preserves the set of changed coordinates. Since the combined coefficient vector is the sum of the four autocorrelation vectors, the total coefficient delta is the sum of the four sequence-level deltas. Applying Theorem 5.2 to each sequence yields (20). □

Theorem 5.4 (Lag-field continuity law with a boundary reservoir)**.** *Let $L = \ell - 1 = 166$. For any transition on the retained graph, define*

$$\rho_t(k) = c_k^{(t)}, \qquad \Delta(k) = \rho_{t+1}(k) - \rho_t(k), \qquad 1 \le k \le L.$$

Let

$$R = \sum_{k=1}^{L} \Delta(k) \tag{21}$$

and define the current

$$J(k) = -\sum_{i=1}^{k} \big(\Delta(i) - R\mathbf{1}\{i = L\}\big), \qquad k = 0, 1, \ldots, L, \tag{22}$$

with the empty sum convention $J(0) = 0$. Then for every $k \in \{1, \ldots, L\}$,

$$\rho_{t+1}(k) - \rho_t(k) = J(k-1) - J(k) + R\mathbf{1}\{k = L\}. \tag{23}$$

Proof. Fix $k \in \{1, \ldots, L\}$. By definition (22),

$$J(k-1) = -\sum_{i=1}^{k-1} \big(\Delta(i) - R\mathbf{1}\{i = L\}\big)$$

and

$$J(k) = -\sum_{i=1}^{k} \big(\Delta(i) - R\mathbf{1}\{i = L\}\big).$$

Subtract the second identity from the first:

$$\begin{aligned} J(k-1) - J(k) &= -\sum_{i=1}^{k-1} \big(\Delta(i) - R\mathbf{1}\{i = L\}\big) + \sum_{i=1}^{k} \big(\Delta(i) - R\mathbf{1}\{i = L\}\big) \\ &= \Delta(k) - R\mathbf{1}\{k = L\}. \end{aligned}$$

Rearranging gives

$$\Delta(k) = J(k-1) - J(k) + R\mathbf{1}\{k = L\}.$$

Since $\Delta(k) = \rho_{t+1}(k) - \rho_t(k)$ by definition, (23) follows. Every step has been explicit, so the theorem is proved. □

Proposition 5.5 (Orbit-signature invariance under reversal and sign flip)**.** *Let two candidate events have the same normalized event kind, action count, support flux, changed-lag count, and transport span, and suppose their delta profiles differ only by reversal and/or global sign inversion. Then they induce the same orbit signature* (16)*.*

Proof. All components of (16) except the final one depend only on the quantities explicitly listed in the statement, so those components agree immediately.

It remains to check the sign-run count $r(e)$. The implementation computes $r(e)$ by extracting the sign sequence of the delta profile and then taking the minimum run-length encoding among the four variants

$$\text{original}, \quad \text{reversed}, \quad \text{sign-flipped}, \quad \text{reversed sign-flipped}.$$

If one profile is obtained from the other by reversal and/or sign inversion, then the two sets of four variants are identical as sets. Therefore the minimum run-length encoding is the same for both events, and hence the run-count component $r(e)$ is the same. Thus all six components of (16) agree, proving the proposition. □

Theorem 5.6 (Depth-1 locality barrier on the canonical frontier seed)**.** *Let $\mathcal{A}_1$ be the explicit actuator class consisting of*

(i) all 334 *one-packet moves,*

(ii) all 55,611 *unordered two-packet moves,*

(iii) all 9 *retained low-splash composite actions.*

Then no actuator in $\mathcal{A}_1$ strictly improves the canonical frontier objective $(13, 2880, 512)$*.*

Proof. The statement is a complete finite computation recorded in `results/verification/radius_limited_locality_barrier.json`. The proof proceeds family by family.

One-packet family. The direct locality scan `results/analysis/frontier_locality_scan.json` exhausts all 334 packets and reports

$$\text{better} = 0, \qquad \text{equal} = 1, \qquad \text{worse} = 333.$$

The unique neutral packet is `s[83]`.

Two-packet family. The barrier checker evaluates all 55,611 unordered two-packet actions by exact delta computation and reports no improving pair. The best pair is the neutral action

$$[\texttt{s[84]}, \texttt{s[166]}],$$

which leaves the objective unchanged.

Retained composite family. The same checker evaluates all 9 retained low-splash actions and again finds no improving single actuator. The best retained action is the same neutral pair as above.

Since every member of $\mathcal{A}_1$ belongs to one of the three families above, and each family has been checked exhaustively with no strict improvement, no actuator in $\mathcal{A}_1$ improves the canonical frontier seed. This proves the theorem. □

Theorem 5.7 (First certified depth-2 local counterexample)**.** *In the retained radius-1, depth-2 causal-cone rule class, there exists a strict local improvement of the canonical frontier seed. The first certified counterexample in lexicographic checker order is the edge*

$$[3,7] = \big([\texttt{q[53]}, \texttt{q[136]}],\ [\texttt{q[29]}, \texttt{s[29]}, \texttt{q[114]}, \texttt{s[114]}]\big), \tag{24}$$

which reaches the six-packet state

$$[\texttt{q[29]}, \texttt{q[53]}, \texttt{q[114]}, \texttt{q[136]}, \texttt{s[29]}, \texttt{s[114]}]$$

with objective $(13, 2744, 480)$.

Proof. The barrier certificate defines the checked rule class explicitly:

(a) both actions must belong to the retained library $\mathcal{R}$,

(b) the two actions must overlap on changed-lag support,

(c) the union of their changed-lag sets must stay within the retained low-splash envelope of at most 25 lags.

The checker then enumerates every admissible ordered causal cone from the canonical seed state.

The action $a_3 = [\texttt{q[53]}, \texttt{q[136]}]$ is itself a depth-1 non-improving retained action with objective

$$(14, 2820, 496).$$

The action $a_7 = [\texttt{q[29]}, \texttt{s[29]}, \texttt{q[114]}, \texttt{s[114]}]$ is also depth-1 non-improving when applied directly to the seed. However, the ordered cone (a_3, a_7) is admissible in the checked rule class and its end state, stored in the barrier JSON as state 53, has objective

$$(13, 2744, 480).$$

To see that this is a strict improvement, compare it lexicographically with the seed objective:

$$13 = 13, \qquad 2744 < 2880, \qquad 480 < 512.$$

Hence the end state has the same support as the seed and strictly smaller ℓ_1 and maximum defect. Therefore it strictly improves the objective.

The checker reports `counterexample_count = 2` and stores the first counterexample exactly as the edge $[3, 7]$ above. Thus the canonical seed is not blocked once the local rule class is widened from depth-1 single actuators to retained radius-1 depth-2 causal cones. This proves the theorem. □

Proposition 5.8 (Artifact-certified failure of the H1 continuation gate)**.** *In the original one-packet frontier batch, H1 fails all continuation conditions:*

(a) no exact hit,

(b) no best-objective improvement over the seed,

(c) no strict support-plus-magnitude advantage over the matched baselines.

Proof. The frontier summary `results/experiments/order_668_64m/summary.json` records

$$\texttt{exact\_hit} = \texttt{false}$$

for H1 and

$$\texttt{best\_objective} = (13, 2880, 512),$$

which is exactly the seed tuple. This proves (a) and (b).

For (c), the same artifact records H1 terminal restart tuples

$$(33, 2368, 384), \quad (40, 2356, 408), \quad (23, 2216, 384),$$

with median terminal tuple $(33, 2356, 384)$. Greedy and simulated annealing each remain at $(13, 2880, 512)$. Therefore H1 does not hold a strict support advantage over those baselines, because $33 > 13$. Hence it fails the continuation gate on support even though some terminal magnitudes are lower. This proves (c). □

6 Experimental Setup

6.1 Inputs and instance families

The paper uses four input families.

1. **Exact controls.** Two deterministic one-flip controls of lengths 5 and 7, corresponding to orders 20 and 28.
2. **Canonical frontier seed.** The recovered order-668 64-modular seed from Eliahou's 2025 paper.
3. **Harder lag-space control.** `control_n9_hardest_pair`, used for the lattice-gas branch and the orbit-rule training mix.
4. **Frontier perturbation ladder.** Five retained-state perturbations (`barrier_ladder_01` through `barrier_ladder_05`) chosen to have no immediate retained-action improvement but at least one improving depth-2 cone.

No external dataset is involved. All experiments are deterministic except simulated annealing, stochastic hillclimbing, and the population branch.

6.2 Shared configurations

H1-era batch. The original control and frontier pilot use shared evaluation budget 80, requested restart count 3, RNG seed 17, and restart packet flips 2. These are the settings stored in `results/experiments/order_668_64m/summary.json`.

Retained-basis branches. The hypergraph, lattice-gas, orbit, and population branches operate on the retained graph with lookup budget 256 per run. The lattice-gas branch uses warning decay 1, boost 2, cap 8, and reservoir cap 16. The orbit branch uses the fixed 137-signature rule table saved in `results/experiments/order_668_orbit_ca/rule_table.json`. The population branch uses the control-only 133-signature table, population size 64, coupling 1.6, temperature 1.2, contraction threshold 0.42, memory mix 0.55, and maximum rounds 6.

6.3 Artifact chronology and provenance discipline

The repository contains one stale draft and later experiments that postdate it. The H1-only paper, methods brief, and review notes were written before the commits

`18a1568`, `f86898e`, `0dce7ad`, `30a278c`, `5cf94fa`.

Those later commits add the retained-state graph, the barrier certificate, the lattice-gas branch, the orbit branch, and the population branch. The present manuscript therefore uses the H1 writeup only for the H1-specific negative control. All post-H1 claims are anchored directly to the raw summaries, JSON logs, and code paths added in the later commit range rather than to the stale memo layer.

6.4 Hardware and reproducibility limits

The saved artifacts do *not* record CPU model, memory size, or operating-system build as first-class experiment metadata. We therefore report wall times exactly as stored, but we do not interpret them as hardware-normalized speed measurements. This is a real reproducibility limitation. In contrast, seeds, budgets, configuration files, and raw result JSONs are explicit.

6.5 Statistics

The paper uses two kinds of evidence.

1. **Exact finite computations.** The one-packet scan, the composite-library scan, the depth-1 barrier, the state-graph construction, and the first certified counterexample are deterministic exhaustive computations. No confidence intervals are appropriate for these quantities.
2. **Sampled stochastic outcomes.** Exact-hit rates on the tiny controls and success/contraction rates for the population branch are reported with Wilson 95% intervals, solely to expose small-sample uncertainty.

7 Results

7.1 H1 is executable, but the original one-packet frontier story is negative

The negative H1 result remains fully valid and still matters. Figure 3 shows that H1 reaches exactness on the two tiny controls, which rules out a trivial implementation failure. Figure 4 and Proposition 5.8 then show the frontier failure: no exact hit, no best-objective gain over $(13, 2880, 512)$, and terminal improvement in ℓ_1 or maximum defect only after large support diffusion.

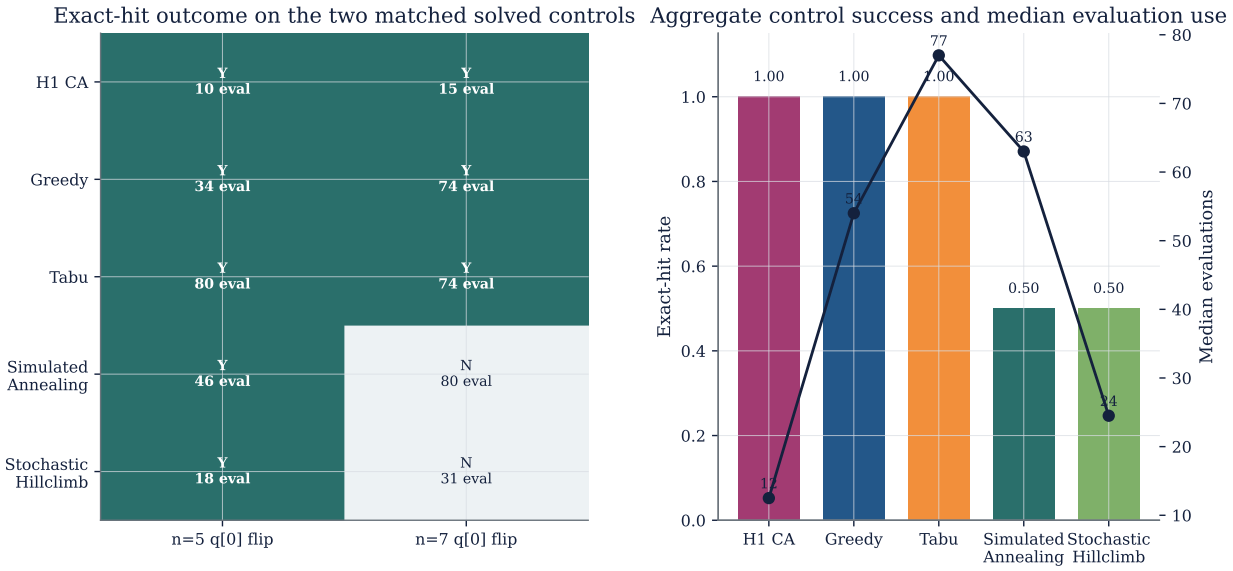

Figure 3: **Matched solved controls for the original H1-era batch.** H1 reaches exactness on both tiny controls, as do greedy and tabu. The control batch therefore validates the harness and the H1 implementation, but it does not establish a CA-specific frontier advantage.

Table 2: **Original H1-era evidence.**

Method / batch	Best tuple	Terminal readout	Exact hits	Notes
H1 on controls	exact on both	0/0/0 on both solved controls	2/2	executable negative control
Greedy on controls	exact on both	0/0/0 on both solved controls	2/2	same-coordinate baseline
H1 on canonical frontier	13/2880/512	median terminal 33/2356/384	0/3	no seed improvement
Greedy on canonical frontier	13/2880/512	13/2880/512	0/1	pinned to seed
Simulated annealing on canonical frontier	13/2880/512	13/2880/512	0/1	pinned to seed

7.2 The one-packet failure sharpens into a retained-library locality barrier

The direct locality scan and the actuator atlas show that the problem is not "CA failed" in the abstract, but that the original actuator basis is fake-local. Figure 6 shows the one-packet scan; the median one-packet move changes 51 lags and no packet improves the objective. Figure 7 then shows what the actuator redesign accomplishes and what it does not. The retained library cuts the median footprint to 9, yet none of its nine depth-1 actions improves the seed.

The exact barrier statement is stronger than the original H1 diagnosis. Theorem 5.6 covers not only the one-packet basis, but the full checked depth-1 class of 55,954 single actuators. The canonical frontier seed is therefore certified frozen for every local rule that fires only one actuator from that class.

7.3 A radius-1 depth-2 hyperedge produces the first certified local improvement

The barrier does not prove that local CA are hopeless. It proves that depth-1 is hopeless in the checked class. Theorem 5.7 then identifies the first escape: the ordered cone $[3, 7]$. Figure 8 visualizes the path and compares the retained-basis branches across the frontier ladder.

The hypergraph CA uses that exact rule class and beats every immediate retained-basis baseline

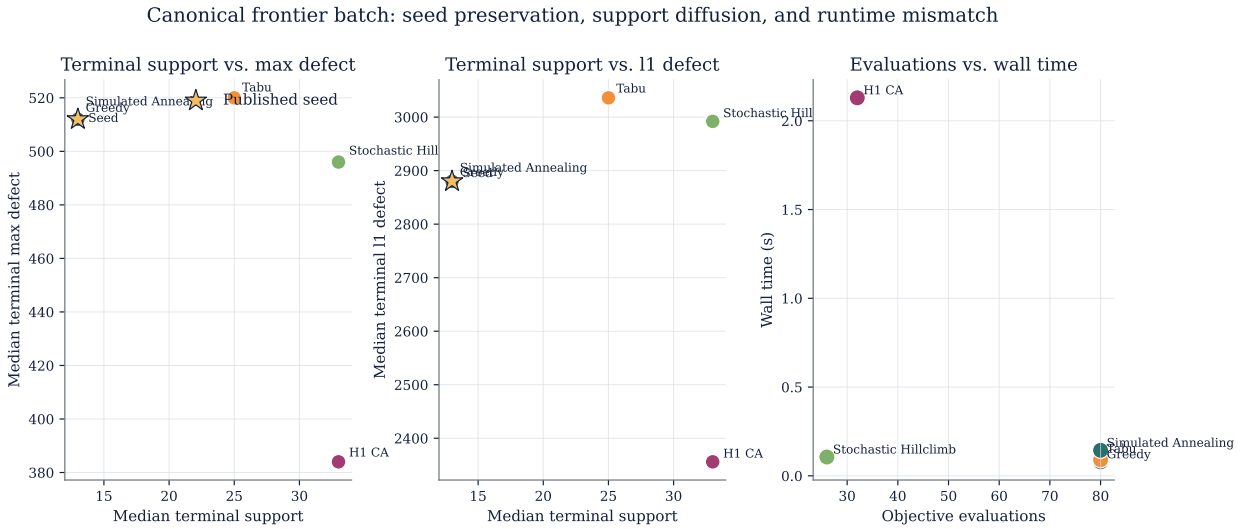

Figure 4: **Canonical frontier batch on the one-packet basis.** The one-packet H1 run never improves the seed best objective, even though some terminal restarts lower ℓ_1 and maximum defect after increasing support far above the seed support 13. Greedy and simulated annealing stay pinned to the seed. This is the negative control that motivates the retained-basis redesign.

Table 3: **Composite-family scan and retained library.**

Family	Candidates	Median changed lags	Low-splash count	Best depth-1 frontier outcome
One-packet	334	51.0	—	neutral only
Pair	55,611	57.0	860	neutral only
Balanced-four	13,861	50.0	362	no neutral, no improvement
Boundary quad	85	34.0	7	no neutral, no improvement
S-boundary pair	84	33.5	9	no neutral, no improvement
Q-boundary pair	3	42.0	0	no neutral, no improvement
Retained Pareto set	9	9.0	9	neutral only

on the canonical seed and on all five ladder states. On the canonical seed, it reaches the six-packet state $(13, 2744, 480)$ in one accepted update. The pairwise, zero-coupling, zero-refractory, and scorer-only baselines all remain stuck at the start state.

7.4 The lattice-gas branch reproduces the same state and passes the family-leakage audit

The hypergraph result might still be dismissed as a graph-local coincidence unless a different representation recovers it. The lattice-gas branch does exactly that. It works on the signed lag field, not on raw packet coordinates, and ranks transitions by the continuity-law signature from Theorem 5.4. On the canonical frontier seed it reaches the same improved state $(13, 2744, 480)$ while both lag-space single-action controls stay at $(13, 2880, 512)$. On the harder control `control_n9_hardest_pair`, the lattice-gas branch reaches exactness $(0, 0, 0)$.

Equally important, the family-leakage audit passes. The improved frontier state shows no palindromic, prime-involution-fixed, cocyclic, block-circulant, or nontrivial periodic collapse in the derived channels. The branch therefore survives as a lag-space reinterpretation of the same local

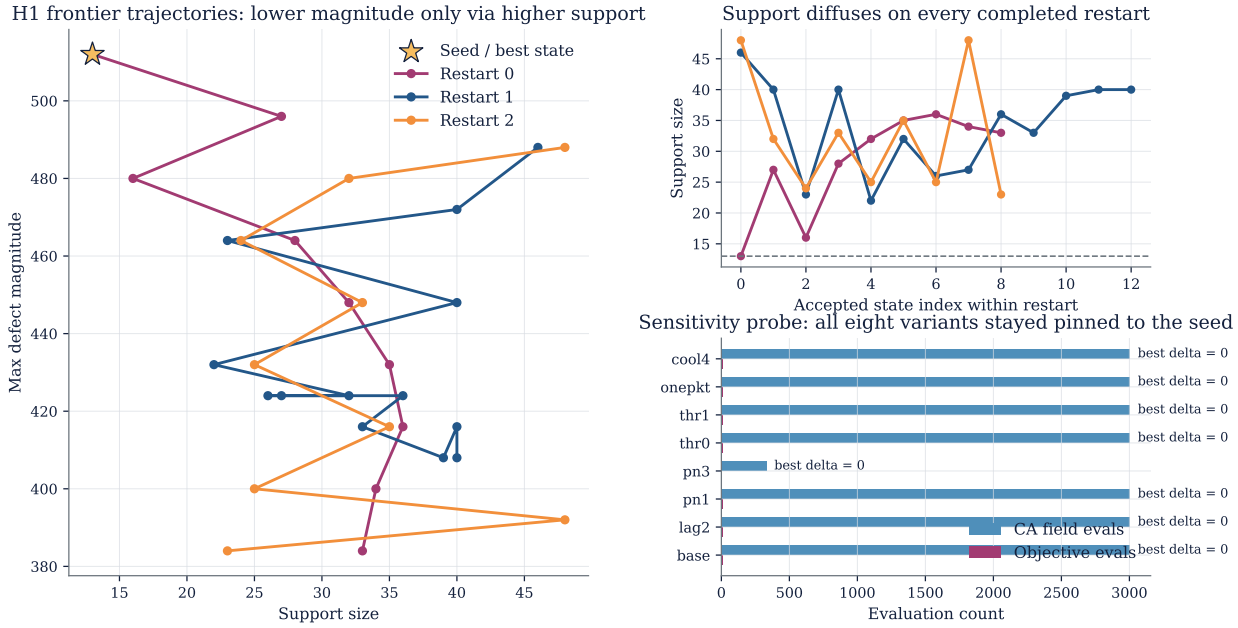

Figure 5: **H1 dynamics on the canonical seed.** The three frontier restarts move toward lower maximum defect only by diffusing support; nearby H1 sensitivity variants in the saved probe remain pinned to the seed best objective. The combination of diffusion and paralysis is the negative-control endpoint of the H1 branch.

Table 4: **Canonical frontier results after actuator redesign.**

Method	Best tuple on canonical seed	Work unit	Accepted updates	Status
Pairwise retained baseline	13/2880/512	9 lookups	0	blocked
Zero-coupling retained baseline	13/2880/512	9 lookups	0	blocked
Zero-refractory retained baseline	13/2880/512	9 lookups	0	blocked
Scorer-only retained baseline	13/2880/512	9 lookups	0	blocked
Hypergraph CA	13/2744/480	210 lookups	1	improves
Lattice-gas CA	13/2744/480	242 lookups	1	improves
Orbit CA	13/2744/480	65 lookups	1	improves
Population median	13/2744/480	5 RNG seeds	median 1	ties zero-coupling
Zero-coupling population median	13/2744/480	5 RNG seeds	median 1	no worse than coupled

escape, not as a disguised search over a known restricted family.

7.5 The orbit quotient transfers the same improvement across the perturbation ladder

The orbit branch tests a stronger claim than the hypergraph or lattice-gas branches: can one fixed local rule table operate on symmetry-quotient classes rather than on raw packet identities? The answer in the saved artifacts is yes, but only for the same non-exact improvement. The orbit rule table has 137 signatures and is trained without the canonical frontier seed itself. It improves all six frontier states shown in Figure 8 and, on the canonical seed, reaches the target state in 65 representative lookups instead of the hypergraph CA's 210.

The orbit result therefore strengthens the repository's final position in one specific way. The retained counterexample is not merely a single coordinate-space trick. It survives quotienting by

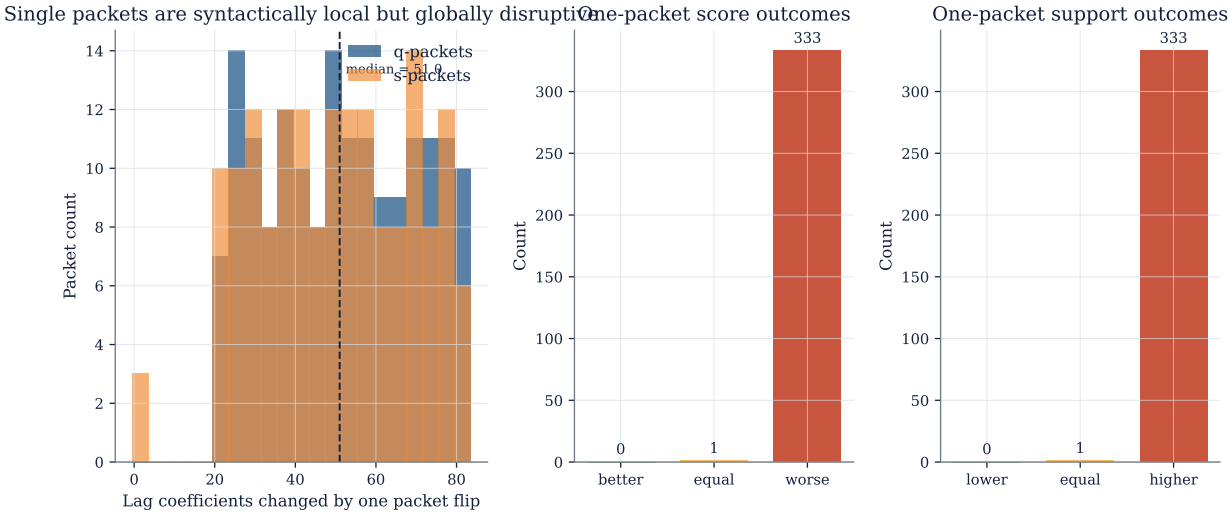

Figure 6: **Direct one-packet locality scan on the canonical frontier seed.** No one-packet move improves the seed objective, only one packet is neutral, and the median footprint is 51 changed lags. This is the exact evidence behind the first layer of the depth-1 barrier.

Table 5: **Lattice-gas CA and matched lag-space controls.**

Instance	Lattice-gas CA	Lag-greedy control	Warning-field control
Canonical frontier	13/2744/480	13/2880/512	13/2880/512
Hard control `n9`	0/0/0	0/0/0	0/0/0
Family leakage audit	pass	not applicable	not applicable

cyclic position, reversal, and global sign orientation through Proposition 5.5. It still does *not* solve order 668, but it does transfer the same frontier improvement beyond one raw-coordinate representation.

7.6 Population self-stabilization fails to beat zero-coupling

The population branch is the main robustness test. It asks whether stochastic coupling over orbit representatives can turn the retained local escape into a stronger self-organizing phenomenon. Figure 9 and Table 6 show that it cannot, at least in the saved regime. On the canonical seed, the coupled and zero-coupling branches both improve in 3/5 runs and both contract in 3/5 runs. Across the perturbation ladder, the coupled population never achieves a strict majority of wins over zero-coupling. The phase map is diffusion-only at every tested coupling/threshold pair.

Figure 7: **Retained composite library on the canonical frontier seed.** Left: median changed-lag counts by actuator family. The low-splash cutoff is set at half the one-packet median, namely 25.5. Only the retained Pareto set reaches median footprint 9. Right: every retained depth-1 action still fails lexicographically. The neutral pair $A0$ leaves the seed unchanged, while $A3$ and $A7$ each improve ℓ_1 but only after increasing support. These two actions later reappear as the first improving depth-2 cone.

8 Discussion

8.1 What the repository now establishes

The strongest answer to the original cellular-automaton question is no longer the stale H1 statement "we tried CA and it failed." The current repository establishes four narrower and stronger facts.

1. The original one-packet CA basis is genuinely blocked on the canonical seed, not merely unpromising.
2. The obstruction is not all local dynamics; it is depth-1 local actuation in the checked class.
3. A redesigned low-splash actuator basis plus depth-2 local coordination yields a reproducible six-packet state that strictly improves the frontier seed to $(13, 2744, 480)$.
4. That same improved state is not tied to one code path: it appears in the hypergraph, lattice-gas, and orbit branches.

These are substantial structural facts even though they stop short of an exact Hadamard matrix.

8.2 What the repository does *not* establish

The paper still does not justify any of the following stronger claims.

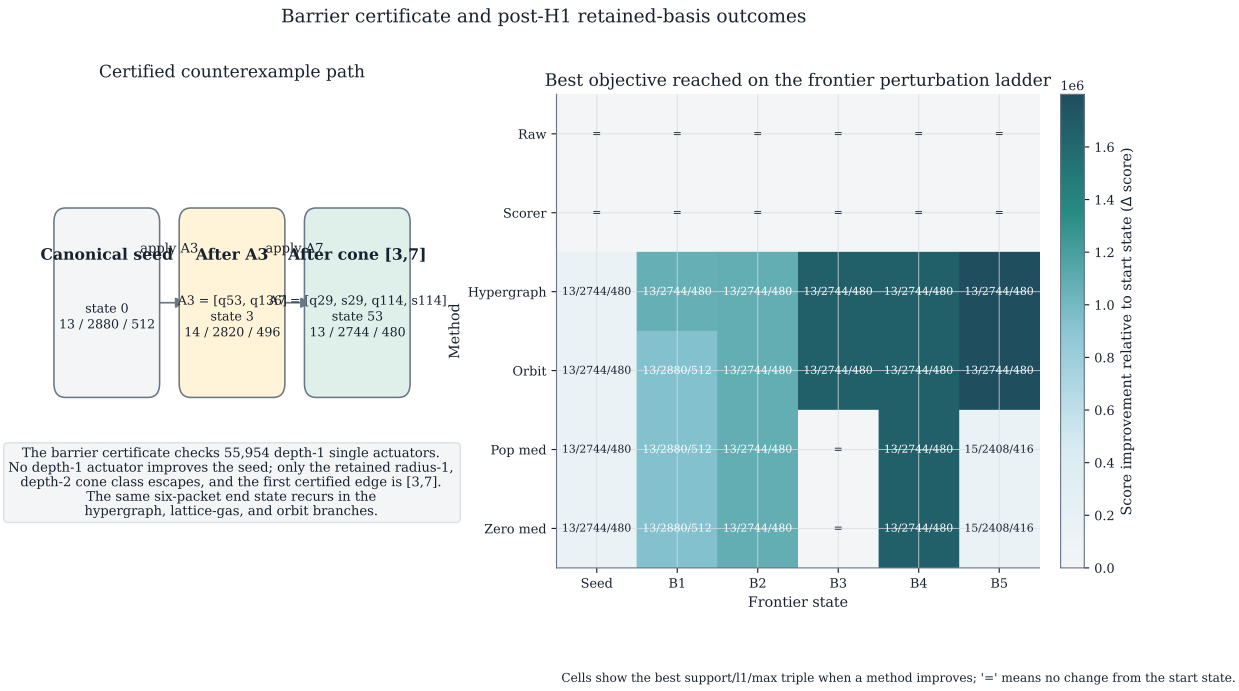

Figure 8: **Barrier certificate and retained-basis branch outcomes.** Left: the first certified counterexample path. The single action $A3$ alone worsens support, but the admissible cone $[3, 7]$ returns support to 13 while lowering both ℓ_1 and maximum defect. Right: best state reached by each retained-basis method on the canonical frontier seed and the five perturbation-ladder states. Hypergraph CA improves all six states. Orbit CA matches or exceeds the same outcomes with far fewer lookups. Population medians improve some states but never separate from zero-coupling.

- Hadamard order 668 is not solved.
- No exact order-668 witness or impossibility proof is produced.
- The retained-basis branches do not prove that cellular automata are broadly competitive with annealing, SAT+CAS, or other Hadamard-search paradigms.
- The current evidence is not a literature-scale benchmark. It is one frontier seed, one retained-library perturbation ladder, and one population robustness sweep.
- The post-H1 branches are supported by direct raw artifacts, but the repository does not contain a second full review-round narrative packet aligned to those later commits. We therefore keep the post-H1 claims empirical and local rather than sweeping and novelty-heavy.

8.3 Failure modes and limitations

Three limitations matter most.

Exactness remains open. All successful retained-basis branches converge to the same non-exact six-packet state. The paper does not know whether that state is a stepping stone toward an exact witness or a local dead end of a different kind.

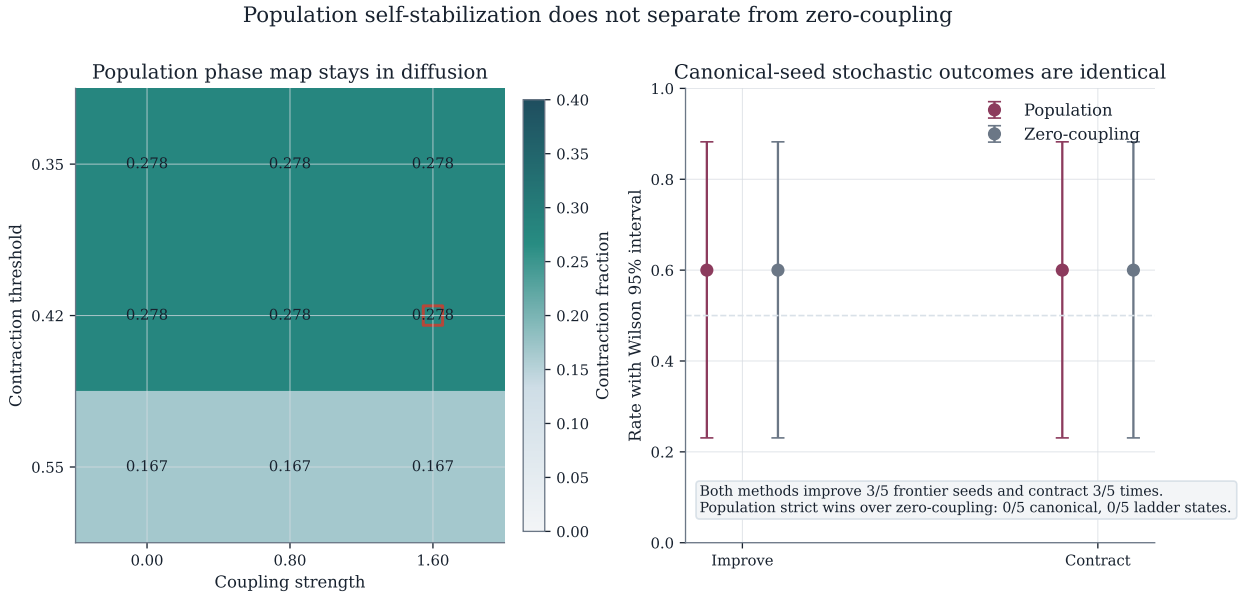

Figure 9: **Population self-stabilization does not separate from zero-coupling.** Left: contraction fraction over the canonical seed and the perturbation ladder for the tested coupling/threshold grid. Every cell remains in the diffusion regime. Right: canonical-seed improvement and contraction rates with Wilson 95% intervals. The coupled and zero-coupling branches are indistinguishable in the saved evidence, and the coupled branch records zero strict wins over zero-coupling on both the canonical seed and the ladder.

The retained library is tiny and hand-fixed by a deterministic rule. The library has only nine actions because the retention rule is deliberately severe. That severity is a strength for barrier certification, but also a weakness: it may exclude other useful local mechanisms that require a slightly larger retained basis.

Hardware provenance is incomplete. The repository preserves configurations and outputs well, but not the machine profile. As a result, work accounting in this paper is lookup- or evaluation-based rather than hardware-normalized.

8.4 Open questions

The current barrier/counterexample pair suggests several concrete next questions.

1. Does any retained-basis local rule reach a state with support below 13?
2. Can a larger but still certifiable low-splash library produce deeper exact escapes without collapsing into global rescoring?
3. Is the six-packet state $(13, 2744, 480)$ unique up to the current equivalence notion inside the retained graph, or are there qualitatively different local escapes?
4. Can the retained-basis search be combined with exact subproblem certification without shifting all credit to the exact layer?

These are the meaningful next steps. More H1 weight tuning is not.

Table 6: **Population branch summary (canonical seed and ladder medians).**

State	Population median	Zero-coupling median	Coupled contractions
Canonical seed	13/2744/480	13/2744/480	3/5
Barrier ladder 01	13/2880/512	13/2880/512	3/5
Barrier ladder 02	13/2744/480	13/2744/480	3/5
Barrier ladder 03	15/2408/416	15/2408/416	1/5
Barrier ladder 04	13/2744/480	13/2744/480	3/5
Barrier ladder 05	15/2408/416	15/2408/416	3/5
Canonical strict wins over zero-coupling		0/5	
Ladder-state strict wins over zero-coupling		0/5	

8.5 Societal and scientific implications

The direct societal stakes are limited, since the target is an abstract combinatorial object. The scientific lesson is about method discipline. Open-problem work on near-solutions is especially vulnerable to rhetoric drifting ahead of exactness. The current repository avoids that failure only when the claim is phrased precisely: one-packet CA fails, retained depth-2 local escape succeeds narrowly, population robustness fails, and the exact order-668 problem remains open.

9 Conclusion

Cellular automata do not solve Hadamard order 668 in this repository. The original H1 one-packet branch is a clean negative control: it reaches exactness on tiny controls but neither improves the canonical frontier seed nor survives the continuation gate. The post-H1 redesign then sharpens, rather than erases, that failure. Exact exhaustive evaluation over 55,954 depth-1 single actuators proves a locality barrier on the canonical seed. The first verified escape appears only at radius 1, depth 2 on the retained overlap graph, reaching the six-packet state $(13, 2744, 480)$. The same state is reproduced by a lag-core lattice-gas CA and transferred by a symmetry-quotient orbit CA, while a population self-stabilizing CA fails to beat zero-coupling.

The honest answer to the user's request is therefore two-part. First, "cellular automata" did not find an exact Hadamard matrix of order 668. Second, a carefully redesigned local CA pipeline did find a certified and reproducible frontier improvement, together with an exact barrier/counterexample description of why the original one-packet approach failed. That is not a solution paper. It is a rigorous negative-plus-narrow-positive structural result on a concrete open order.

A Additional Proof Details

A.1 State-count lemma for the retained graph

Lemma A.1. *The retained-state constructor produces exactly* 512 *states on the canonical frontier seed.*

Proof. There are 9 retained actions, so there are exactly $2^9 = 512$ subsets of $\mathcal{R}$. The con-

structor enumerates all 512 subset masks and maps each mask to its parity packet mask. By definition, the number of retained states is the number of unique parity packet masks encountered. The saved summary `results/experiments/order_668_hypergraph_ca/summary.json` reports `unique_state_count = 512`. Since the number of unique states cannot exceed the number of subset masks and both are 512, every subset mask induces a distinct parity state. Therefore the retained graph contains exactly 512 states. □

A.2 Why the lattice-gas current is nearest-neighbor

Theorem 5.4 defines the current by cumulative summation. This automatically makes $J(k)$ a quantity living on the edge between lags k and $k+1$. The continuity equation then expresses every lag update $\Delta(k)$ as incoming current minus outgoing current, plus a single boundary-reservoir term at the final lag. No hidden long-range transport term is left over, because the cumulative definition absorbs the entire non-boundary part of the update into the telescoping difference $J(k-1) - J(k)$.

B Additional Experimental Tables

B.1 Full retained-basis ladder outcomes

Table 7: **Frontier perturbation ladder outcomes for the retained-basis branches.** Hypergraph and orbit use the exact best states stored in their summary JSONs. Population and zero-coupling use the saved medians.

State	Start tuple	Hypergraph	Orbit	Population median	Zero median
Canonical frontier	13/2880/512	13/2744/480	13/2744/480	13/2744/480	13/2744/480
Barrier ladder 01	14/2812/496	13/2744/480	13/2880/512	13/2880/512	13/2880/512
Barrier ladder 02	14/2820/496	13/2744/480	13/2744/480	13/2744/480	13/2744/480
Barrier ladder 03	15/2408/416	13/2744/480	13/2744/480	15/2408/416	15/2408/416
Barrier ladder 04	15/2408/416	13/2744/480	13/2744/480	13/2744/480	13/2744/480
Barrier ladder 05	15/2544/448	13/2744/480	13/2744/480	15/2408/416	15/2408/416

B.2 Population canonical-seed rates with Wilson intervals

Table 8: **Canonical-seed population rates.**

Method	Improvement rate	Wilson 95% CI	Contraction rate
Population	3/5	[0.23, 0.88]	3/5
Zero-coupling	3/5	[0.23, 0.88]	3/5

B.3 Per-run H1 control outcomes

Table 9: **Per-run H1-era control outcomes.**

Seed	Method	Exact hit	Terminal support	Max defect	Evaluations
control_n5_q0	H1 CA	yes	0	0	10
control_n5_q0	Greedy	yes	0	0	34
control_n5_q0	Simulated annealing	yes	0	0	46
control_n5_q0	Stochastic hillclimb	yes	0	0	18
control_n5_q0	Tabu	yes	0	0	80
control_n7_q0	H1 CA	yes	0	0	15
control_n7_q0	Greedy	yes	0	0	74
control_n7_q0	Simulated annealing	no	1	4	80
control_n7_q0	Stochastic hillclimb	no	1	4	31
control_n7_q0	Tabu	yes	0	0	74

C Implementation Details and Reproduction Notes

C.1 Saved file anchors

The most important direct artifacts for reproducing the narrative are:

- `results/frontier/order_668_64m/seed_manifest.json`,
- `results/analysis/frontier_locality_scan.json`,
- `results/analysis/composite_packet_locality_atlas.json`,
- `results/analysis/composite_packet_retained_library.json`,
- `results/verification/radius_limited_locality_barrier.json`,
- `results/experiments/order_668_hypergraph_ca/summary.json`,
- `results/experiments/order_668_lattice_gas/summary.json`,
- `results/experiments/order_668_orbit_ca/summary.json`,
- `results/experiments/order_668_population_ca/summary.json`,
- `results/verification/family_leakage_audit.json`.

C.2 Code paths

The relevant implementation files are:

- `hadamard_ca/harness.py` for shared exactness metrics,
- `hadamard_ca/h1_ca.py` for the H1 negative control,
- `hadamard_ca/composite.py` for composite-action enumeration and exact deltas,

- `hadamard_ca/retained_state_graph.py` for retained-state construction,
- `hadamard_ca/lag_lattice_gas.py` for transport signatures and lattice-gas policies,
- `hadamard_ca/orbit_ca.py` for the orbit quotient,
- `hadamard_ca/population_ca.py` for the population branch.

C.3 Figure generation

The H1-era figures are preserved from `scripts/generate_paper_artifacts.py`. The post-H1 figures in this paper are generated by `scripts/generate_post_h1_figures.py`. Both scripts output PDF and PNG at 600 DPI.

C.4 Minimal rerun commands

The direct rerun commands are the repository scripts already used to create the saved artifacts:

- `python3 scripts/run_h1_ca.py`
- `python3 scripts/run_baseline.py`
- `python3 scripts/generate_composite_locality_atlas.py`
- `python3 scripts/check_radius_limited_locality_barrier.py`
- `python3 scripts/run_hypergraph_ca.py`
- `python3 scripts/run_lattice_gas_ca.py`
- `python3 scripts/run_orbit_ca.py`
- `python3 scripts/run_population_ca.py`
- `python3 scripts/generate_post_h1_figures.py`

D Hyperparameter and Budget Snapshot

Table 10: **Saved hyperparameter snapshot.**

Item	Value
H1 shared budget	80 evaluations
H1 requested restarts	3
H1 RNG seed	17
H1 restart packet flips	2
H1 lag neighborhood radius	1
H1 packet neighborhood radius	2
H1 same-channel / cross-channel weights	0.15/0.35
H1 activation threshold	0.5
H1 refractory steps / penalty	2/0.75
Retained low-splash cutoff	25.5 changed lags
Retained action count	9
Retained state count	512
Retained-branch lookup budget	256
Lattice-gas reservoir cap	16
Orbit rule-table size	137 signatures
Population control-only table	133 signatures
Population size	64
Population coupling / threshold	1.6/0.42
Population temperature / memory mix	1.2/0.55
Population RNG seeds	$\{11, 13, 17, 19, 23\}$

References

[1] Curtis Bright, I. Kotsireas, and Vijay Ganesh. A sat+cas method for enumerating williamson matrices of even order. *AAAI Conference on Artificial Intelligence*, 2018. doi: 10.1609/aaai.v32i1.12203. URL https://www.semanticscholar.org/paper/eec5b2d83d3853c24cf070b4701fc2c86b594222.

[2] Curtis Bright, Dragomiru Z. Dokovic, I. Kotsireas, and Vijay Ganesh. The sat+cas method for combinatorial search with applications to best matrices. *Annals of Mathematics and Artificial Intelligence*, 2019. doi: 10.1007/s10472-019-09681-3. URL https://www.semanticscholar.org/paper/74aeec834364032b323d56b8a1c03c798fcd2844.

[3] Matteo Cati and Dmitrii V. Pasechnik. A database of constructions of hadamard matrices. *arXiv.org*, 2024. doi: 10.48550/arXiv.2411.18897. URL https://www.semanticscholar.org/paper/9f2616d562a9810dfc55db9f9acf606c2ef62296.

[4] S. Eliahou and M. Kervaire. Modular sequences and modular hadamard matrices. *Journal of Combinatorial Designs*, 2001. doi: 10.1002/JCD.1007. URL https://www.semanticscholar.org/paper/74c6c98807c49280d7dbdc8c935f625788248b03.

[5] S. Eliahou and M. Kervaire. A survey on modular hadamard matrices. *Discrete Mathematics*, 2005. doi: 10.1016/j.disc.2005.02.021. URL https://www.semanticscholar.org/paper/b0bc1d196cbba14824aebcbb2410226ff2688d3d.

[6] Shalom Eliahou. A 64-modular hadamard matrix of order 668. *Australasian Journal of Combinatorics*, 93(2):422–427, 2025. URL https://ajc.maths.uq.edu.au/pdf/93/ajc_v93_p422.pdf.

[7] M. Gadouleau, Luca Mariot, and S. Picek. Bent functions from cellular automata. *IACR Cryptology ePrint Archive*, 2020. URL https://www.semanticscholar.org/paper/792b2568f1335f14e5282dd837943d109fbb4b85.

[8] Luca Mariot, Maxime Gadouleau, Eric Formenti, and Alessio Marcuzzi. Mutually orthogonal latin squares based on cellular automata. *Designs, Codes and Cryptography*, 88(2):391–411, 2020. doi: 10.1007/s10623-019-00689-8. URL https://link.springer.com/article/10.1007/s10623-019-00689-8.

[9] A. B. Suksmono. Finding a hadamard matrix by simulated quantum annealing. *Entropy*, 2018. doi: 10.3390/e20020141. URL https://www.semanticscholar.org/paper/d98246df005a52ff9a5e99b6c5f548c1def1ba12.

[10] A. B. Suksmono and Yuichiro Minato. Finding hadamard matrices by a quantum annealing machine. *Scientific Reports*, 2019. doi: 10.1038/s41598-019-50473-w. URL https://www.semanticscholar.org/paper/c42e6a8a231cd8cd638ff953b6aa7b539c4596e9.

[11] A. B. Suksmono and Yuichiro Minato. Quantum computing formulation of some classical hadamard matrix searching methods and its implementation on a quantum computer. *Scientific Reports*, 2022. doi: 10.1038/s41598-021-03586-0. URL https://www.semanticscholar.org/paper/1fd50d1147b7684686e533886b44a9e92ac2e807.

[12] Michail-Antisthenis I. Tsompanas, Nikolaos I. Dourvas, Konstantinos Ioannidis, Georgios Ch. Sirakoulis, Reinhard Hoffmann, and Andrew Adamatzky. Cellular automata applications in shortest path problem. In Andrew Adamatzky and Georgios Ch. Sirakoulis, editors, *Cellular Automata and Discrete Complex Systems*, volume 10872 of *Lecture Notes in Computer Science*, pages 119–134. Springer, 2018. doi: 10.1007/978-3-319-77510-4_8. URL https://link.springer.com/chapter/10.1007/978-3-319-77510-4_8.